

Transfer Learning for Appliance Manipulation from Guidebooks with Failure-Triggered Human Feedback

Anonymous Submission

Abstract—Robots often encounter unseen appliances in new environments. User manuals and guidebooks contain rich procedural, semantic, and dynamics information that can accelerate robot learning on these devices. However, grounding guidebook information in the real world is challenging: robots must map guidebook entities to observations, convert instructions into actions, and interpret described dynamics to monitor progress and detect failures. We present BookWorm, a framework for learning manipulation tasks from guidebooks using on-demand human feedback that is distilled into transferable knowledge for new devices. Our key insight is that guidebooks provide structural priors for both action generation and environmental dynamics. BookWorm uses these priors to initialize a joint policy-forward model refined via actively queried human feedback after self-detected failures, distilling knowledge that enables efficiently learning novel appliances. Experiments on real-world appliance manipulation demonstrate that BookWorm leverages guidebook information more effectively than prior approaches and learns related, novel devices more efficiently with less human feedback. See additional materials on our project site, bookworm-robot.github.io.

I. INTRODUCTION

Consider a housekeeper in an unfamiliar home tasked with cooking and cleaning with a variety of appliances. To operate a previously unseen device, they would naturally consult its user manual rather than uninformed trial-and-error. We envision robot housekeepers that can also read guidebooks to rapidly learn new tasks. Given the vast corpus of manuals, repair guides, and operating procedures, guidebooks represent a rich and largely untapped source of knowledge for accelerating robot learning in new environments.

The central challenge is **grounding** guidebook knowledge in the robot’s perception, action, and state spaces. At the perceptual level, the robot must associate textual and diagrammatic entities in the guidebook, such as buttons, handles, and indicators, with objects in its observations. At the action level, it must translate high-level procedural instructions into executable robot motions. Beyond perception and control, the robot must also understand the dynamics described in the guidebook (e.g., pressing a particular button illuminates an indicator or opens a compartment) to reason about task progress and detect failures during execution. While human feedback can be queried to resolve such failures, it is inefficient to require similar corrections for each new appliance. To scale guidebook-driven robot learning, the robot must not only ground guidebook knowledge on a single device, but also transfer knowledge from previous devices to reduce supervision when adapting to new ones.

Our key insight is that guidebooks provide structural priors over entity functions, action procedures, and dynamics, and



Fig. 1. BookWorm learns to operate new household devices by leveraging rich information from guidebooks. To ground guidebook information to the real world, BookWorm actively queries humans for feedback to update its policy, which enables it to learn new devices and guidebooks with much less human supervision.

that these priors can be refined by a human and transferred to efficiently learn from new guidebooks of related devices. We propose BookWorm, **Bootstrapping Object-grounded Operational Knowledge With On-demand Robot Mentoring**, a framework that extracts these priors from a guidebook, uses their predicted dynamics to detect grounding and execution failures, and solicits human feedback only then. By distilling the corrections into reusable policy representations, the robot progressively improves at grounding new guidebooks while requiring less supervision on each.

To realize this framework, BookWorm (see Figure 1) integrates symbol grounding, skill synthesis, and guidebook-conditioned dynamics understanding. Rather than predicting robot trajectories directly, we represent behavior hierarchically: a high-level policy selects skills and parameters, while code-based skills realize these decisions as motions. This factorization enables feedback to update individual skills, which then carry over to new devices with minimal corrections.

We evaluate BookWorm on five real-world household appliances across both single-device settings and multi-device transfer settings. Our experiments demonstrate that guidebook grounding improves task performance by 30 percentage points over the next best baseline, guidebook-

conditioned dynamics prediction improves state tracking and failure detection, and distilled human feedback on prior devices enables more efficient adaptation to novel devices.

We present three contributions: 1) BookWorm, the first framework for learning real-world robot manipulation from guidebooks with human feedback; 2) an active-querying loop that detects grounding and execution failures and solicits multimodal corrections; 3) a transfer procedure that reduces human supervision when learning unseen, related appliances.

II. RELATED WORK

Robot learning from guidebooks and textual guidelines.

Closest to our setting, prior work has explored using furniture assembly manuals for **real robot tasks** by grounding visual diagrams and pictorial instructions in visual observations [1–3]. However, these works do not tap into the large reservoir of textual information in guidebooks, such as step-by-step procedural information and environmental dynamics descriptions. ApBot [4] learns symbolic appliance models from manuals but focuses only on button pressing and knob turning. Unlike our work, it does not consider how to learn or refine manipulation skills from guidebook information.

A broader body of work studies sequential decision making in **simulated domains**, where agents act from textual guidelines. Prior methods learn policies from game manuals [5, 6], ground entities and dynamics in gridworld environment descriptions [7–10], or enforce language-specified constraints during RL training [11–14]. Unlike these settings, we study grounding guidebooks in real-world, long-horizon manipulation tasks, where dynamics are more stochastic and exhaustive RL exploration is infeasible. We instead treat the guidebook as a structured prior over perception, action, and task dynamics. Our setting also resembles rule-constrained real-world systems [15–17], but these works do not address how to synthesize skills from guidebooks, actively query human feedback, or efficiently transfer to new domains.

Human-in-the-loop Robot Learning. DROC [18] and LYRA [19] distill iterative human corrections into a library of code policies, and Vocal Sandbox [20] learns skill functions and environment entities from multimodal human feedback. Unlike these methods, which assume constant human supervision or focus on skill acquisition from feedback, we use guidebooks as the initial source of structured task knowledge and actively query humans only when grounding or skill failures occur. Our query mechanism is related to opportunistic active learning [21, 22], active predicate learning [23], query arbitration [24], and conformal prediction [25, 26]. While these methods query based on uncertainty over predefined symbols or classifiers, BookWorm queries when observed outcomes do not match guidebook-conditioned state predictions, enabling it to address failures in perception, skill execution, or skill selection.

Transfer and Continual Learning. We study the transfer learning setting where knowledge gained in one domain can be cross-applied to more efficiently learn in a second domain [27]. We refer the reader to representative prior

works in subfields of transfer learning, including sim-to-real [28–31] and real-to-sim-to-real [32, 33] transfer, cross-embodiment transfer [34–37], and multitask transfer [38–40]. None of these works have explored how guidebooks can aid in efficient transfer to new devices and domains. Our transfer setting also differs from continual learning benchmarks [41] and approaches that mitigate catastrophic forgetting through parameter regularization [42], replay [43], modular subnetworks [44], continually growing skill latent libraries [45–47], or large pretrained VLAs [48, 49]. Instead of mastering long task streams and minimizing catastrophic forgetting, we focus on forward transfer on a target appliance by accumulating a symbolic skill library, grounding-related corrections, and distilled human language corrections.

Code-based policies and reward functions. We build on executable programs as policies [50–56] and reward functions [57–60]. Code naturally supports skill composition, flexible parameterization, and language-based refinement. Prior works on code-based policies focus on semantic generalization across pick-and-place tasks, and those on reward functions convert natural language feedback to actionable RL goals. Neither addresses the core grounding challenge of synthesizing skills and policies from rich prior knowledge in guidebooks.

Language-conditioned dynamics and predictive robot models. Prior work studies how language improves representations and predictions in latent dynamics and visual world models [61–65]. Instead of learning an expensive pixel-space world model, BookWorm uses textual dynamics information from guidebooks to improve action predictions and solicit human feedback when actual dynamics deviate from these specifications.

III. PROBLEM SETUP

We study guidebook-conditioned robot manipulation. For each device D , the robot is given a guidebook G and a task instruction l , such as “Make a light brown waffle,” or “Disable the thermostat schedule.” We focus on using instructions, diagrams, and textual dynamics information from guidebooks to execute manipulation tasks, leaving broader use of safety and troubleshooting information to future work.

At each timestep t , the robot receives an image observation I_t , state vector s_t , and a textual interaction history h_t . The state s_t is composed of end-effector pose, joint positions, force measurements, and gripper state, while the history h_t contains previous language descriptions of states, executed actions, and any human feedback received so far. We denote the task context as $c = (G, l)$. The goal is to complete the task specified by l on appliance D while minimizing the amount of human feedback required.

We model robot behavior hierarchically. The high-level policy selects a skill and its parameters:

$$\pi : I_t \times h_t \times c \times \Omega \mapsto (\omega_t, \theta_t),$$

where Ω is the current skill library, $\omega_t \in \Omega$ is the selected skill, and θ_t contains the parameters needed to instantiate that skill. This high-level skill-parameter pair is the action space

of the decision-making problem studied in this paper. The robot state s_t is used by the selected skill implementation rather than by the high-level policy.

The selected skill is then executed by a low-level code-based skill implementation. Each skill maps its parameters and the robot state to a sequence of executable robot motions:

$$\omega_t : \theta_t \times s_t \mapsto X_t,$$

where $X_t = \{x_{t,i}\}_{i=1}^k$ is a variable-length sequence of Cartesian end-effector poses. Thus, the high-level policy decides *what* skill to execute and with *which* parameters, while the low-level skill implementation determines *how* to realize that decision as robot motion.

Some skill parameters correspond to symbolic guidebook entities, such as buttons, handles, lids, indicators, or compartments. These parameters must be grounded before policy training. We use Θ_G to represent the set of parameters requiring spatial grounding. For each appliance D , the robot must construct a device-specific grounding map:

$$\Gamma_D : \Theta_G \rightarrow \mathbb{R}^3,$$

which maps guidebook entities to 3D locations in the robot workspace. Since appliance geometry and layout vary across devices, these spatial coordinates are device-specific and must be estimated for each new appliance.

Actively Queried Human Feedback. During training, the robot may query a human for feedback, which can be in freeform language l_t^H to correct behavior, or visual annotations (e.g., keypoints or polygonal regions) to correct symbol groundings and failure detection regions. To reduce human supervision burden, we assume the human does not initiate feedback and may only provide it when proactively queried by the robot. During evaluation, no human feedback is available.

Single-task and Transfer Learning. We consider two learning settings. In the *single-device setting*, the robot learns to perform a task on one appliance D using its guidebook and queried human feedback. In the *transfer learning setting*, the robot first learns on a source appliance D_s and then adapts to a target appliance D_t from the same appliance category. The target appliance differs in geometry, layout, interface, and articulation, but its shared functional mechanics enable the transfer of useful manipulation skills, recovery behaviors, and human-guided corrections learned from the source appliance. The objective is to more efficiently learn the task on D_t with less human feedback based on experience gained from D_s .

Transferring to the target device does not entail simply reusing fixed spatial coordinates from the source device. Instead, the target device must be re-grounded using its own guidebook and observations. BookWorm leverages reusable knowledge from the source device (i.e., skill implementations and distilled human feedback) to guide skill synthesis, grounding, and high-level decision making.

IV. BOOKWORM

BookWorm instantiates the hierarchical problem setup from Section III via a guidebook-grounded policy refined

through human feedback and transferred across devices (see Figure 2). Given guidebook G and task instruction l , BookWorm builds an initial policy prior by synthesizing executable skills, identifying parameters requiring grounding, mapping guidebook entities to segmentation masks, and predicting expected state changes per action. When observed outcomes diverge from guidebook-conditioned expectations, BookWorm queries a human for corrective feedback and updates the policy accordingly.

The key design principle is *reusable feedback*. Corrections fix current failures and are distilled into a knowledge base. To operate a new device more efficiently, we leverage the knowledge base and skills from previous devices to establish strong policy and grounding priors. To instantiate this framework, we rely on off-the-shelf vision-language-models. Symbol grounding maps guidebook diagrams to open-vocabulary segmentation masks (GSAM-2), refined with human annotations where necessary. Skill synthesis, policy-forward prediction, failure detection, and feedback integration are all performed via GPT-5.4 in our experiments.

A. Guidebook-Grounded Initialization

BookWorm converts the guidebook into a policy prior over skills, grounded parameters, and expected state transitions. First, BookWorm synthesizes the **skill library** Ω based on the task context $c = (G, l)$. This process decomposes high-level guidebook instructions into semantically meaningful code-based skills (e.g., opening a lid, inserting a filter) that are implemented via a low-level robot API (see Section V-F). BookWorm must decide the skill functions and their corresponding parameter spaces to achieve task context c . We use code-based skill representations as a structured interface between guidebook language and robot control because they are synthesizable from instructions, editable via language feedback, and structurally reusable across devices.

Second, to ground skill parameter entities Θ_G , BookWorm constructs a **grounding** Γ_D mapping each $\theta_g \in \Theta_G$ to an xyz coordinate. GSAM-2 [66–68] runs on robot observations using queries from Θ_G , producing candidate masks backprojected into point clouds. The symbol grounder then reasons over these masks, guidebook diagrams G_d , and entities Θ_G to assign 3D locations. Diagrams disambiguate visually identical parts distinguishable only by guidebook labels. For any unresolvable entities, BookWorm queries a human for keypoint or polygon annotations. Keypoints are processed by GSAM2 to generate segmentation masks, whereas polygons act as direct masks; both are subsequently backprojected and filtered to extract an xyz centroid coordinate for θ_g .

Third, with an initial skill library Ω in place, the **high-level policy** selects skill-parameter pairs (ω_t, θ_t) to execute at the current timestep t . Inspired by unified VLA architectures that function as both action and next state predictors [69–75], the policy additionally predicts the expected post-execution next state in language, $\hat{l}_{s,t+1}$:

$$\pi_{pf} : I_t \times h_t \times c \times \Omega \rightarrow (\omega_t, \theta_t, \hat{l}_{s,t+1}),$$

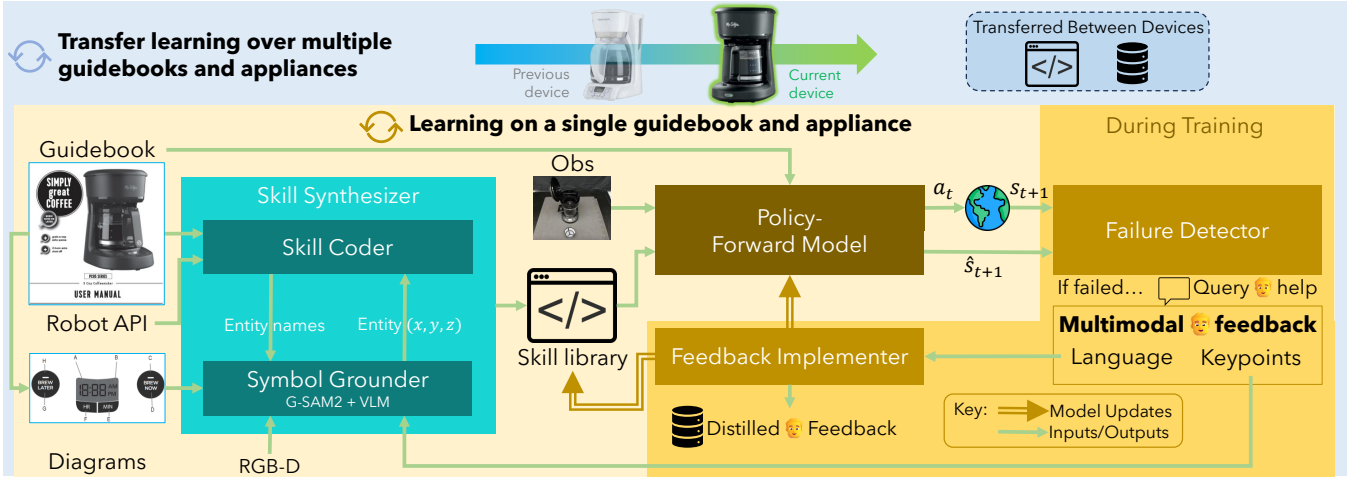


Fig. 2. **System Overview.** BookWorm initializes a guidebook-grounded policy by synthesizing skills, grounding entities, and predicting expected next states. During training, it detects failures, queries humans for feedback, updates the policy and skill library ($\Rightarrow$) using this feedback, and distills it into reusable knowledge. When adapting to a new device, BookWorm leverages this reusable knowledge, alongside the guidebook and visual observations of the target appliance, to regroup its entities and skills.

By leveraging the guidebook to jointly predict actions and language-based next states, π_{pf} enables BookWorm to query human feedback on-demand when actual environmental dynamics deviate from its predictions.

B. Refinement with Failure-Triggered Multimodal Feedback

The initial guidebook-grounded policy is often imperfect, with spatially imprecise skills and incorrect high-level action predictions. To refine the policy, BookWorm *actively solicits* targeted human feedback, crucially removing the need for constant human supervision while enabling continual policy refinement. After executing high-level action $a_t = \omega_t(\theta_t)$, a failure detector evaluates whether the observed next state matches the predicted one:

$$f_d : I_t \times a_t \times I_{t+1} \times \hat{I}_{s,t+1} \rightarrow [0, 1],$$

which returns the probability of action a_t resulting in an execution failure $p_{\text{fail}} \in [0, 1]$.

When p_{fail} exceeds a threshold τ (50% in our experiments), the robot queries the human for **language feedback** l_t^H , ranging from skill-specific (e.g., “push the button harder”) to high-level (e.g., “close the lid before changing the time”). The feedback implementer automatically applies corrections by scope. Skill-specific feedback updates Ω , and general feedback enriches the policy context. In parallel, skill and policy corrections are distilled into knowledge base $\mathcal{D}_H$ for reuse on new, within-class devices.

Detecting failures requires f_d to compare the forward model’s predicted next state $\hat{I}_{s,t+1}$ (e.g., “the brew light is on”) against the observed image I_{t+1} . This comparison often hinges on small visual cues, such as whether an indicator LED has changed, which a VLM frequently misses or assesses with poorly calibrated confidence. During training, we therefore allow the human to correct f_d whenever p_{fail} falls within δ of τ . The human does so through **visual annotation feedback**: a region of the control panel (e.g., a rectangle or oval around a small indicator LED) that

f_d should focus on, optionally accompanied by language feedback describing what to look for there (e.g., “whether a green light has turned on/off”). Each skill $\omega \in \Omega$ maintains its own annotation and accompanying language feedback. Because device placement and wrist-camera viewpoint vary slightly across trials, we use ORB feature matching [76] and RANSAC-estimated homography [77] to relocate the annotated region from the annotation image to the pre- and post-action image observations I_t and I_{t+1} . Both are cropped before being passed to f_d .

C. Transfer Across Devices

Appliances in the same category (e.g., waffle makers) often share task structure and manipulation strategies despite differences in geometry, layout, interface, and articulation. BookWorm exploits this by transferring two components from a source appliance D_s to a target appliance D_t : (1) distilled human feedback in $\mathcal{D}_H$ from the source appliance, and (2) the skill library Ω .

BookWorm then grounds the target guidebook to obtain Γ_{D_t} and conditions the policy on this grounding and the distilled feedback. To account for differences in geometry and morphology, BookWorm uses the target guidebook, distilled feedback, and Γ_{D_t} to generate target-specific variants of the source-device skills. The policy then selects among these skills based on the current observation and target guidebook, reducing the human feedback needed on D_t .

V. EXPERIMENTAL SETUP



Fig. 3. We experiment with various household devices in both the single-device and transfer settings.

A. Devices and Tasks

We evaluate our method on a real-world Franka Emika Panda Arm in both the *single device* setting and the *transfer learning* setting. Guidebooks are downloaded as PDFs, with only the English-language pages processed for efficiency. For the *single device* setting, we evaluate on a **Honeywell thermostat** (X2P-RTH20B) where the task is to disable schedule settings by navigating multiple menus, a **Mr. Coffee maker** (PC05) to load a filter, brew coffee, and adjust the time, and an **Amazon Basics waffle maker** (AB-258NA) to make a light brown waffle. To evaluate *transfer learning*, we experiment with two device pairs. In the coffee maker pair, we first train on Mr. Coffee, then transfer the learned policy to a morphologically distinct **Black+Decker coffee maker** (CM1160-W), on the same coffee brewing and time adjusting task. In the waffle maker pair, we first train on an **Amazon Basics waffle maker** (AB-258NA), then transfer to a **Chefman waffle maker** (RJ04-V3), on the same waffle making task. See Figure 3 for our experimental appliances.

On the **thermostat**, the schedule setting starts switched on and the display is on the home screen. The robot must press MENU → SELECT → DOWN → SAVE & EXIT → EXIT. On the **Mr. Coffee** maker, the robot must (1) insert the filter into the brew basket, (2) close the lid, (3) press BREW NOW, and (4) press HR twice. We experiment with the same task on the **Black+Decker** maker but it is considerably larger, has more buttons with different functions and indicator lights, and has a lid hinged along a different axis. On the **Amazon Basics** waffle maker, the robot must (1) pour batter onto the bottom plate, (2) close the lid, (3) turn the knob to LIGHT, and (4) turn it back to OFF. We evaluate on the same task on the **Chefman** waffle maker, which has different knob angles for each setting, handle shape, and a significantly higher bottom plate height.

B. Training and Evaluation Procedure

For each method on each device, we train for five trials, during which the human can be queried for corrective natural language feedback, before evaluating for ten trials. All trials are at most 12 timesteps, and all methods emit an empty action to terminate the trial. After providing feedback, the human operator manually resets the scene to the pre-action configuration if an irrecoverable failure has occurred, allowing the robot to retry. During evaluation, all modules are frozen and operate without human supervision. Consequently, the failure detector is disabled, though the policy continues to predict the next state to improve skill and parameter selection.

C. Metrics

We evaluate three key metrics: (1) full task success rate, an all-or-nothing metric measuring the proportion of trials where all steps were completed in order, (2) step completion rate, a partial credit metric measuring the fraction of consecutive steps completed in order, and (3) the number of human feedback rounds, a measure of human training effort.

D. Baselines

In the *single device setting*, we compare against **Code-as-Policies (CaP)** [50] and **Promptbook** [52]. CaP generates skill functions directly from a guidebook but does not extract action or dynamics information from it and does not solicit human feedback. Promptbook extends CaP with more concrete skill-implementation guidelines and human feedback: it queries a human when it detects a lack of progress and incorporates episode-level feedback. Like our method, it updates its skill functions continually across episodes.

In the *transfer-learning setting*, we compare against **Experience Replay** [43], a continual-learning method that mitigates catastrophic forgetting by replaying data from earlier tasks while learning the current one. We adapt it to our VLM-based, in-context setting using the human feedback our method accumulated during training. Skill-level feedback is inserted into the context of the skill implementer, and policy-level feedback into the joint policy-forward model. Unlike our method, there is no failure-triggered human feedback. We evaluate two variants: **Experience Replay** uses feedback from both source and target devices, and **Experience Replay, Single Device** uses only target-device feedback. All baselines receive the same observations as our method.

E. Ablations

In the *single device setting*, we test four ablations of our method: not using the guidebook for planning or skill synthesis (**No guidebook**), not predicting the next state with the policy (**No next state pred.**), and not having human language feedback (**No human feedback**). In the *transfer learning setting*, we ablate the value of skills learned and feedback accumulated from the source appliance by training everything from scratch on the target appliance (**Ours, Single Device**). We also ablate the value of training on the target appliance (**Ours, Zero-shot, Device 2**).

F. Robot Base API

To synthesize skills from guidebooks, the robot can call roughly 10 functions from a provided base API. These fall into three groups: (1) perception and state queries that return Cartesian coordinates, such as `get_xyz(obj_name)`, which returns object positions from RGB-D observations, and `get_ee_pos()`, which returns the gripper position; (2) gripper actions, such as `open_gripper()`, `close_gripper()`, `set_gripper_state(pos)` (which sets the width of the parallel-jaw gripper); and (3) arm motions, such as `move_to_xyz(xyz)`, `move_to_pose(xyz, euler)`, `move_in_dir_until_force(xyz_dir, force)`, and `rotate_wrist(angle)`. Consequently, the synthesized skills can be arbitrary-length pose waypoint sequences defined relative to object locations, with force-terminated motions for contact-rich steps such as pressing buttons.

VI. EXPERIMENTAL RESULTS

To evaluate our framework, we seek to answer four research questions (RQ1 – RQ4), presented as headings in

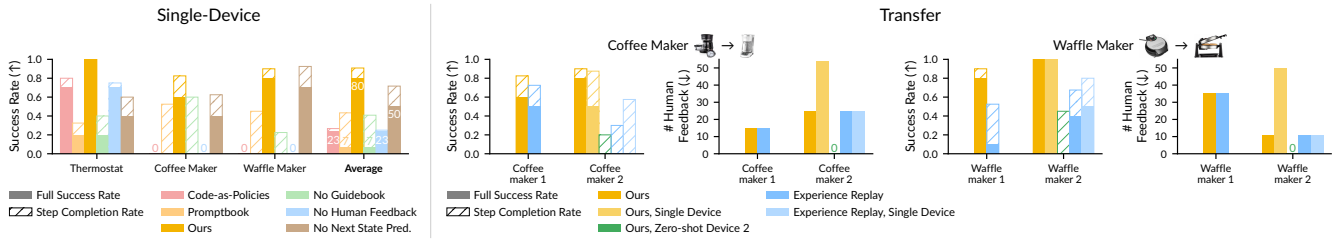


Fig. 4. **Top row:** In the single device setting, our method (BookWorm) outperforms prior work baselines and all ablations on both devices. **Bottom two rows:** In the transfer learning setting, our method achieves comparable success as ablations and baselines when learning the first device, but greatly reduces human effort compared to a single-device ablation learned solely on the second device. Each bar represents 10 real-world trials.

the following subsections. Figure 4 visualizes the results for RQ1 and RQ2 in the *single device setting* (left), and RQ3 and RQ4 in the *transfer learning setting* (right). Across all conditions on success rate graphs (10 trials per bar), solid bars denote full task success, while hollow, diagonally-striped bars represent step completion rates. For statistical significance and reported p -values, we use the exact Cochran-Mantel-Haenszel (CMH) test for success rate and the stratified Wilcoxon-Mann-Whitney test (van Elteren) for step completion rate and human effort.

(RQ1) Does BookWorm more effectively learn appliance operation tasks from guidebooks compared to prior works?

We first compare BookWorm to CaP [50] and Promptbook [52] to measure how effectively each grounds policies in guidebooks. BookWorm achieves 80% full success and 91% step completion, far exceeding CaP (23%, 27%) and Promptbook (7%, 43%). These results are statistically significant ($p < 10^{-6}$) for both methods and both metrics. We attribute the overperformance of BookWorm to the fact that neither baseline explicitly grounds the described actions and appliance dynamics in the physical world, which our ablations in RQ2 confirm.

(RQ2) How important are the guidebook, human feedback, and forward model for our appliance manipulation tasks?

Removing the guidebook from the policy and skill synthesizer (**No Guidebook**) decreases the average single device success rate of BookWorm from 80% to 7% ($p = 1.4 \times 10^{-9}$), confirming that guidebook grounding is necessary for both skill synthesis and sequential planning. However, the guidebook alone is not sufficient for enabling performant policies; it is also crucial to solicit human feedback for refining guidebook-initiated policies. Our method without human feedback averages 23% ($p = 4.1 \times 10^{-7}$), succeeding only on the thermostat where human feedback is not needed to synthesize simple button-pressing skills.

In BookWorm, the policy jointly predicts the action and the resulting next state in language. Removing next-state prediction from the policy drops performance from 80% to 50% ($p = 0.028$), demonstrating that actively grounding guidebook dynamics in real-world transitions enables BookWorm to both predict better actions and detect failures that warrant human feedback.

(RQ3) Can BookWorm efficiently learn on new devices by transferring knowledge and skills from previous devices within the same category?

In the transfer setting, we measure how much less effort BookWorm needs to learn a new target device from its guidebook after learning a source device. Evaluated zero-shot on the target device (**Ours, Zero-shot Device 2**), BookWorm achieves 0% success rate and 23% step completion on average, showing that learning on the target device is necessary. Source-device knowledge does not transfer directly because the target device differs substantially in layout, geometry, and articulation.

Compared to learning the target device from scratch (**Ours, Single Device**), BookWorm achieves, on average, a 15 percentage point (pp) higher success rate on the target device with 54% to 78% fewer rounds of human feedback ($p = 0.0094$). This demonstrates that our transfer mechanism, which distills generalizable language feedback and refines skill functions adapted from the previous device, enables learning new within-class devices while significantly reducing device-specific human supervision.

(RQ4) How does BookWorm compare to prior transfer-learning approaches in guidebook-conditioned manipulation tasks?

To compare transfer ability independent of feedback quantity and quality, we condition **Experience Replay** on the same human feedback given to BookWorm. Averaged over both target devices, BookWorm outperforms Experience Replay by 70 pp in success rate ($p = 3.9 \times 10^{-6}$) and 46 pp in step completion rate ($p = 9.5 \times 10^{-6}$). Restricting Experience Replay to target-device feedback only (**Experience Replay, Single Device**) narrows these gaps to 65 pp ($p = 1.2 \times 10^{-5}$) and 26 pp ($p = 4.2 \times 10^{-5}$), respectively, indicating that feedback from other devices hinders its adaptation. In contrast, BookWorm benefits from prior devices: its modular design allows the distilled knowledge and skill library to carry over with minimal refinement.

VII. CONCLUSION

We presented BookWorm, which learns real-world appliance manipulation tasks from guidebooks by grounding entities and synthesizing executable skills. For policy refinement and efficient transfer to new devices, BookWorm also uses dynamics information from guidebooks to detect failures

and query humans for multimodal feedback. Empirically, BookWorm better utilizes guidebook information than prior works and greatly decreases the amount of human effort needed to learn new devices within the same class. We believe BookWorm opens the door to leveraging the large corpus of human-written domain information to help robots more efficiently learn physical tasks in real households, and that it can substantially improve sample efficiency over robot learning frameworks that do not use guidebooks.

Limitations and Future Work. We only explored code-based skill representations. While easy to extract from guidebooks and refine with language feedback, they often lack expressivity needed for articulated appliances that require precise end-effector poses. Future work could pair code-based skills for simple actions such as pick-and-place with IL/RL-trained policies for precise, contact-rich tasks such as insertion, further bridging textual guidebooks and low-level control. BookWorm also primarily uses guidebooks for skill synthesis, planning, and dynamics understanding; future work could further exploit them to learn symbolic reward functions for progress monitoring, or to derive safety constraints that keep robots out of hazardous situations the guidebooks warn against.

REFERENCES

- [1] C. Tie et al., “Manual2skill: Learning to read manuals and acquire robotic skills for furniture assembly using vision-language models,” *arXiv preprint arXiv:2502.10090*, 2025.
- [2] C. Tie et al., “Manual2skill++: Connector-aware general robotic assembly from instruction manuals via vision-language models,” *arXiv preprint arXiv:2510.16344*, 2025.
- [3] J. Zhang, A. Cherian, C. Rodriguez, W. Deng, and S. Gould, “Manual-pa: Learning 3d part assembly from instruction diagrams,” in *ICCV*, 2025, pp. 6304–6314.
- [4] J. Zhang, H. Zhang, A. Xiao, and D. Hsu, “Robot operation of home appliances by reading user manuals,” *arXiv preprint arXiv:2505.20424*, 2025.
- [5] S. Branavan, D. Silver, and R. Barzilay, “Learning to win by reading manuals in a monte-carlo framework,” *JAIR*, vol. 43, pp. 661–704, 2012.
- [6] Y. Wu, Y. Fan, P. P. Liang, A. Azaria, Y. Li, and T. M. Mitchell, “Read and reap the rewards: Learning to play atari with the help of instruction manuals,” *NeurIPS*, vol. 36, pp. 1009–1023, 2023.
- [7] A. W. Hanjie, V. Y. Zhong, and K. Narasimhan, “Grounding language to entities and dynamics for generalization in reinforcement learning,” in *ICML*, 2021, pp. 4051–4062.
- [8] K. Narasimhan, R. Barzilay, and T. Jaakkola, “Grounding language for transfer in deep reinforcement learning,” *JAIR*, vol. 63, pp. 849–874, 2018.
- [9] A. Nguyen and S. Lee, “Language-conditioned world model improves policy generalization by reading environmental descriptions,” *arXiv preprint arXiv:2511.22904*, 2025.
- [10] V. Zhong, T. Rocktäschel, and E. Grefenstette, “Rtfm: Generalising to novel environment dynamics via reading,” *arXiv preprint arXiv:1910.08210*, 2019.
- [11] T.-Y. Yang, M. Y. Hu, Y. Chow, P. J. Ramadge, and K. Narasimhan, “Safe reinforcement learning with natural language constraints,” *NeurIPS*, vol. 34, pp. 13 794–13 808, 2021.
- [12] Z. Wang, M. Fang, T. Tomilin, F. Fang, and Y. Du, “Safe multi-agent reinforcement learning with natural language constraints,” in *AAAI*, vol. 40, 2026, pp. 38 007–38 015.
- [13] B. Prakash, N. R. Waytowich, A. Ganesan, T. Oates, and T. Mohsenin, “Guiding safe reinforcement learning policies using structured language constraints,” in *SafeAI@AAAI*, 2020, pp. 153–161.
- [14] X. Lou, J. Zhang, Z. Wang, K. Huang, and Y. Du, “Safe reinforcement learning with free-form natural language constraints and pre-trained language models,” *arXiv preprint arXiv:2401.07553*, 2024.
- [15] Y. Bai et al., “Constitutional ai: Harmlessness from ai feedback,” *arXiv preprint arXiv:2212.08073*, 2022.
- [16] A. Censi et al., “Liability, ethics, and culture-aware behavior specification using rulebooks,” in *ICRA*, IEEE, 2019, pp. 8536–8542.
- [17] M. F. Ginting et al., “Saycomply: Grounding field robotic tasks in operational compliance through retrieval-based language models,” in *ICRA*, IEEE, 2025, pp. 13 730–13 736.
- [18] L. Zha et al., “Distilling and retrieving generalizable knowledge for robot manipulation via language corrections,” in *ICRA*, IEEE, 2024, pp. 15 172–15 179.
- [19] Y. Meng, Z. Sun, M. Fest, X. Li, Z. Bing, and A. Knoll, “Growing with your embodied agent: A human-in-the-loop lifelong code generation framework for long-horizon manipulation skills,” *arXiv preprint arXiv:2509.18597*, 2025.
- [20] J. Grannen, S. Karamcheti, S. Mirchandani, P. Liang, and D. Sadigh, “Vocal sandbox: Continual learning and adaptation for situated human-robot collaboration,” *arXiv preprint arXiv:2411.02599*, 2024.
- [21] J. Thomason, A. Padmakumar, J. Sinapov, J. Hart, P. Stone, and R. J. Mooney, “Opportunistic active learning for grounding natural language descriptions,” in *CoRL*, PMLR, 2017, pp. 67–76.
- [22] A. Padmakumar, P. Stone, and R. Mooney, “Learning a policy for opportunistic active learning,” in *EMNLP*, 2018, pp. 1347–1357.
- [23] A. Li and T. Silver, “Embodied active learning of relational state abstractions for bilevel planning,” *arXiv preprint arXiv:2303.04912*, 2023.
- [24] K. Bullard, A. L. Thomaz, and S. Chernova, “Towards intelligent arbitration of diverse active learning queries,” in *IROS*, IEEE, 2018, pp. 6049–6056.
- [25] M. Zhao, H. Admoni, R. Simmons, A. Ramdas, and A. Bajcsy, “Conformalized interactive imitation learning: Handling expert shift and intermittent feedback,” in *ICLR*, vol. 2025, 2025, pp. 9238–9251.
- [26] A. Z. Ren et al., “Robots that ask for help: Uncertainty alignment for large language model planners,” *CoRL*, 2023.
- [27] M. E. Taylor and P. Stone, “Transfer learning for reinforcement learning domains: A survey,” *JMLR*, vol. 10, pp. 1633–1685, 2009.
- [28] L. Da, J. Turnau, T. P. Kutralingam, A. Velasquez, P. Shakerian, and H. Wei, “A survey of sim-to-real methods in rl: Progress, prospects and challenges with foundation models,” *ArXiv*, vol. abs/2502.13187, 2025.
- [29] E. Salvato, G. Fenu, E. Medvet, and F. A. Pellegrino, “Crossing the reality gap: A survey on sim-to-real transferability of robot controllers in reinforcement learning,” *IEEE Access*, vol. 9, pp. 153 171–153 187, 2021.
- [30] W. Zhao, J. P. Queralta, and T. Westerlund, “Sim-to-real transfer in deep reinforcement learning for robotics: A survey,” *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, pp. 737–744, 2020.
- [31] A. Yu, A. Foote, R. Mooney, and R. Martín-Martín, “Natural language can help bridge the sim2real gap,” in *RSS*, 2024.
- [32] X. Li et al., “Robogsim: A real2sim2real robotic gaussian splatting simulator,” *ArXiv*, vol. abs/2411.11839, 2024.
- [33] M. M. L. Torné et al., “Reconciling reality through simulation: A real-to-sim-to-real approach for robust manipulation,” *ArXiv*, vol. abs/2403.03949, 2024.

- [34] L. Y. Chen, K. Hari, K. Dharmarajan, C. Xu, Q. Vuong, and K. Goldberg, "Mirage: Cross-embodiment zero-shot policy transfer with cross-painting," *ArXiv*, vol. abs/2402.19249, 2024.
- [35] J. Yang, C. Finn, and D. Sadigh, "Data analogies enable efficient cross-embodiment transfer," *ArXiv*, vol. abs/2603.06450, 2026.
- [36] J. Yang et al., "Pushing the limits of cross-embodiment learning for manipulation and navigation," *ArXiv*, vol. abs/2402.19432, 2024.
- [37] O.-X.-E. Team, "Open x-embodiment: Robotic learning datasets and rt-x models : Open x-embodiment collaboration," *ICRA*, pp. 6892–6903, 2023.
- [38] E. Jang et al., "BC-z: Zero-shot task generalization with robotic imitation learning," in *CoRL*, 2021.
- [39] Y. Jiang et al., "Vima: General robot manipulation with multimodal prompts," *arXiv*, 2022.
- [40] A. Yu and R. J. Mooney, "Using both demonstrations and language instructions to efficiently learn robotic tasks," *arXiv preprint arXiv:2210.04476*, 2022.
- [41] B. Liu et al., "Libero: Benchmarking knowledge transfer for lifelong robot learning," *ArXiv*, vol. abs/2306.03310, 2023.
- [42] J. Kirkpatrick et al., "Overcoming catastrophic forgetting in neural networks," *Proceedings of the national academy of sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.
- [43] D. Rolnick, A. Ahuja, J. Schwarz, T. Lillicrap, and G. Wayne, "Experience replay for continual learning," *NeurIPS*, vol. 32, 2019.
- [44] A. A. Rusu et al., "Progressive neural networks," *arXiv preprint arXiv:1606.04671*, 2016.
- [45] W. Wan, Y. Zhu, R. Shah, and Y. Zhu, "Lotus: Continual imitation learning for robot manipulation through unsupervised skill discovery," in *ICRA*, IEEE, 2024, pp. 537–544.
- [46] J. Xu and X. Nie, "Speci: Skill prompts based hierarchical continual imitation learning for robot manipulation," *IEEE Transactions on Cognitive and Developmental Systems*, vol. 18, pp. 488–503, 2025.
- [47] Y. Yao et al., "Think small, act big: Primitive prompt learning for lifelong robot manipulation," *CVPR*, pp. 22 573–22 583, 2025.
- [48] J. Hu et al., "Simple recipe works: Vision-language-action models are natural continual learners with reinforcement learning," *arXiv preprint arXiv:2603.11653*, 2026.
- [49] H. Liu, C. Kim, B. Liu, M. Liu, and Y. Zhu, "Pre-trained vision-language-action models are surprisingly resistant to forgetting in continual learning," *arXiv preprint arXiv:2603.03818*, 2026.
- [50] J. Liang et al., "Code as policies: Language model programs for embodied control," in *ICRA*, 2023, pp. 9493–9500.
- [51] I. Singh et al., "Progprompt: Generating situated robot task plans using large language models," *arXiv preprint arXiv:2209.11302*, 2022.
- [52] M. G. Arenas et al., "How to prompt your robot: A prompt-book for manipulation skills with code as policies," in *ICRA*, IEEE, 2024, pp. 4340–4348.
- [53] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, "Voxposer: Composable 3d value maps for robotic manipulation with language models," *CoRL*, 2023.
- [54] Y. Mu et al., "Robocodex: Multimodal code generation for robotic behavior synthesis," *ArXiv*, vol. abs/2402.16117, 2024.
- [55] J. Wu et al., "Tidybot: Personalized robot assistance with large language models," *Autonomous Robots*, 2023.
- [56] C. Li et al., "Chain of code: Reasoning with a language model-augmented code emulator," R. Salakhutdinov et al., Eds., vol. 235, ICML, 2024.
- [57] Y. J. Ma et al., *Eureka: Human-level reward design via coding large language models*, 2024. arXiv: 2310.12931 [cs.RO].
- [58] W. Yu et al., *Language to rewards for robotic skill synthesis*, 2023. arXiv: 2306.08647 [cs.RO].
- [59] T. Xie et al., "Text2reward: Reward shaping with language models for reinforcement learning," in *ICLR*, 2023.
- [60] L. Wang et al., "Gensim: Generating robotic simulation tasks via large language models," *ArXiv*, vol. abs/2310.01361, 2023.
- [61] J. Lin et al., "Learning to model the world with language," *arXiv preprint arXiv:2308.01399*, 2023.
- [62] D. Chen et al., "Planning with reasoning using vision language world model," *arXiv preprint arXiv:2509.02722*, 2025.
- [63] J. Gkountouras, M. Lindemann, P. Lippe, E. Gavves, and I. Titov, "Language agents meet causality—bridging llms and causal world models," in *ICLR*, vol. 2025, 2025, pp. 37 311–37 336.
- [64] A. Zhang, K. Nguyen, J. Tuyls, A. Lin, and K. Narasimhan, "Language-guided world models: A model-based approach to ai control," in *Proceedings of the 4th Workshop on Spatial Language Understanding and Grounded Communication for Robotics (SpLU-RoboNLP 2024)*, 2024, pp. 1–16.
- [65] N. Dainese, P. Marttinen, and A. Ilin, "Reader: Model-based language-instructed reinforcement learning," in *EMNLP*, 2023, pp. 16 583–16 599.
- [66] S. Liu et al., "Grounding dino: Marrying dino with grounded pre-training for open-set object detection," *ECCV*, 2024.
- [67] T. Ren et al., "Grounding dino 1.5: Advance the 'edge' of open-set object detection," 2024. arXiv: 2405.10300 [cs.CV].
- [68] N. Ravi et al., *Sam 2: Segment anything in images and videos*, 2024. arXiv: 2408.00714 [cs.CV].
- [69] H. Wu et al., "Unleashing large-scale video generative pre-training for visual robot manipulation," in *ICLR*, vol. 2024, 2024, pp. 10 641–10 662.
- [70] J. Cen et al., "Ryannvla-002: A unified vision-language-action and world model," *arXiv preprint arXiv:2511.17502*, 2025.
- [71] C. Zhu, R. Yu, S. Feng, B. Burchfiel, P. Shah, and A. Gupta, "Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets," *arXiv preprint arXiv:2504.02792*, 2025.
- [72] J. Zhang, Y. Guo, Y. Hu, X. Chen, X. Zhu, and J. Chen, "Up-vla: A unified understanding and prediction model for embodied agent," *arXiv preprint arXiv:2501.18867*, 2025.
- [73] Y. Shen et al., "Videovla: Video generators can be generalizable robot manipulators," *NeurIPS*, vol. 38, pp. 95 597–95 621, 2026.
- [74] Y. Tian et al., "Predictive inverse dynamics models are scalable learners for robotic manipulation," in *ICLR*, vol. 2025, 2025, pp. 92 033–92 052.
- [75] K. Wang et al., "Vagen: Reinforcing world model reasoning for multi-turn vlm agents," *NeurIPS*, vol. 38, pp. 172 871–172 933, 2026.
- [76] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, "ORB: An efficient alternative to SIFT or SURF," in *ICCV*, 2011, pp. 2564–2571.
- [77] M. A. Fischler and R. C. Bolles, "Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.